Purpose/Functionality

A 1-2 sentence description of the purpose or functionality of your site
CHEMCalc is a chemistry study website designed to help students with stoichiometry calculations. It allows students to calculate percent yield, identify limiting reactants, and calculate theoretical yield. It also provides a formula reference page and calculation history so students can review their work.
State Dataclass Attributes

Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass.

Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.
actual_yield: float -> Stores the actual yield entered by the student. Changes when a new percent yield calculation is performed.
theoretical_yield: float: -> Stores the theoretical yield entered or calculated by the student. Changes when a new theoretical yield is calculated.
limiting_reactant: float -> Stores the name of the reactant identified as limiting. Updates when the student completes a limiting reactant calculation.
calculations: list[Calculation] -> Stores previous calculations. A new Calculation object is added whenever the student completes a calculation.
current_page: str -> Stores the name of the current website page. Updates whenever the student navigates to another page.
Second Dataclass

Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)
Dataclass: Calculation
calculation_type: str -> Stores the type of calculation, such as percent yield, limiting reactant, or theoretical yield. Set when a calculation is completed.
result: float -> Stores the numerical answer produced by the calculation. Set when a Calculation object is created.

description: str -> Stores a short explanation of the calculation, such as the values used or the reactant identified. Set when a Calculation object is created.
Routes (minimum 7)

The names of your route functions
home(): Displays the main menu and links to the other pages.
percent_yield(): Displays the percent yield calculator page.
calculate_percent_yield(): Calculates percent yield using the student's actual and theoretical yield inputs. Saves the result in calculation history.
limiting_reactant(): Displays the limiting reactant calculator page.
calculate_limiting_reactant(): Compares the reactant ratios to identify the limiting reactant. Saves the result in calculation history.
theoretical_yield(): Displays the theoretical yield calculator page.
calculate_theoretical_yield(): Calculates theoretical yield using the limiting reactant and product information. Saves the result in calculation history.
formula_reference(): Displays important chemistry formulas and explanations.
Pages (minimum 5)

A brief description of the pages in your site
Home Page - Displays the CHEMCalc title, a short introduction, and buttons linking to the three calculators, and formula reference.
Percent Yield Page- Contains input boxes for actual yield and theoretical yield. Students click a calculate button to see their percent yield result.
Limiting Reactant Page- Allows students to enter the available amounts and balanced equation coefficients of two reactants. Displays which reactant is limiting.
Theoretical Yield Page- Allows students to enter the limiting reactant's mass, molar mass, reactant coefficient, product coefficient, and product molar mass. Displays the theoretical yield.
Formula Reference Page- Displays chemistry formulas for percent yield, converting grams to moles, identifying limiting reactants, and calculating theoretical yield.
Code this to these requirments:Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.

- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the State dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.
    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (global constantsLinks to an external site. are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

More creative and impressive designs are given flexibility on the above requirements - impress us! However, impressive websites tend to do more than the above requirements. Further, if you miss more than 2 requirements, you will get no credit for that submission. For example, missing a second dataclass means you would be missing not only that requirement, but also the list or dictionary of such dataclass instances, and the loop iterating over that list. This would mean that the submission would get a zero.

Code Requirements

Make sure your program and website runs; broken programs do not earn points. This means that if your program contains a syntax error or runs and doesn't deploy a website, you get zero points for this assignment, and are subject to any other penalties listed in the syllabus. Once you are finished with your project code, check over the following:

- Chosen good variable names (check last two sections of Chapter 1B1Links to an external site.)
- Unit tested your functions where possible, being valid and thoroughLinks to an external site.: each route must have at least one unit test, if possible (routes that have randomization are difficult to unit test, so they are not required to).
    - If a function can't be unit tested, you must note as much in a docstring or comment.

- Broke up excessively long lines (80 characters is a good rule of thumb, but we'll be strict about not exceeding 120 characters)
- Included types in your function headers
- Included good docstrings Links to an external site. as the first line of your function's body
- Decomposed complicated functions with helper functions

Your TA will be grading your code's quality based on the ideas above.
Use techniques from python bakery textbook. do not use more advanced code unless 100% necessary

I've written `chemcalc.py` as one Drafter file following your plan. I couldn't run it here, so run it once in your environment before submitting.

How it meets the requirements

- Routes: 10 in total: `index`, `percent_yield`, `calculate_percent_yield`, `limiting_reactant`, `calculate_limiting_reactant`, `theoretical_yield`, `calculate_theoretical_yield`, `formula_reference`, `history` and `filter_history`.
- Pages: 6 pages: home, the three calculators, formulas and history.
- State and second dataclass: `State` has your 5 fields, and `Calculation` has `calculation_type`, `result` and `description`. Every state field gets modified in some route.
- Inputs: many `TextBox`es, plus a `SelectBox` that filters the history.
- Ifs: input validation, the over-100% warning, the limiting-reactant comparison and the history filter.
- Loops: `history_lines` loops over `state.calculations` and reads `calculation_type`. `all_positive_numbers` also loops over the input strings.
- Style: type hints and docstrings are on every function. Long calculations are split into helpers, lines stay under 80 characters, and there are no globals except the constant lists/messages.
- Advanced techniques: none are used, so nothing needs extra documentation. It sticks to bakery-style code: `str` methods, lists, loops and `assert_equal`.

Changes from your plan

- `home()` is now `index()`: Drafter starts on a route named `index`, so I used that name for the home page.

- `limiting_reactant` is a `str`: you listed it as a `float`, but it stores a reactant name, so I made it a `str`.
- Extra route: `history` and `filter_history` (plus the `SelectBox`) cover the "calculation history" feature from your description.
- Validation: I added an error page for bad input, so the site doesn't crash when a student types letters or leaves a box empty.
- Inputs are strings: the route parameters are `str`, and I convert them to numbers only after validating them.

Unit tests
Every route has at least one `assert_equal` that checks the returned page's `.state`. Helpers are tested too. If your version of Drafter doesn't expose `.state` on a `Page`, tell me and I'll adjust the tests.

Possible problem: the limiting reactant page asks for amounts in moles, to keep the math simple. If your assignment expects grams, I can add a molar-mass box for each reactant.

Chemcalc

PY

there shouldnt be a home button on the home page only when you access another page

The home page in `chemcalc.py` no longer shows a Home button, but every other page still does. I split the navigation into two helpers: `page_buttons()` holds the five buttons that go to the other pages, and `nav_buttons()` adds Home in front of them. The home page uses `page_buttons()`, and everything else keeps using `nav_buttons()`. I haven't run it here, so check the home page once in your environment.

Chemcalc

PY

redo docstrings with args and return values if possible

Every function and both dataclasses in `chemcalc.py` now have a docstring with a one-line summary, an `Args:` section with each parameter's name, type and meaning, and a `Returns:` section. The two dataclasses list their fields under `Attributes:` instead. I checked that the file parses as valid Python, but I haven't run the site itself, so give it a quick run in your environment.

- Routes: each docstring says which state fields the route changes. For the calculate routes, `Returns` names both the result page and the error page.
- `add_calculation`: it returns `None` and changes the history list in place, so its `Returns` says that.
- `error_page`: the `back_route` parameter is a function, and I left its type off to match the original header. Add `Callable` yourself if your TA wants a type there.
- No other changes: the code, the tests and the earlier Home-button fix are the same as before.

Chemcalc

PY

are my units tests valid and through coverign all possibilites needed

Mostly valid, but not thorough yet. Every route has a test, so you meet the "one per route" rule, but several branches and some functions are untested. I haven't run any of them, and the route tests depend on one assumption (below).

Validity

- Route tests: they read `.state` on the returned `Page`. I believe Drafter pages have that attribute, but I haven't checked it against your installed version. Run the file once. If it errors on `.state`, tell me and I'll switch the tests to compare whole `Page` objects.
- Weak assertion: the `filter_history` test checks that `calculations == []`. A route that never touched the list would pass it too, so it proves almost nothing.
- Error-case tests: they only check that the history is empty. They should also check that `actual_yield`, `theoretical_yield` and `limiting_reactant` are unchanged.

Functions with no test

- `add_calculation`
- `nav_buttons` and `page_buttons`
- `error_page`
- `history_page`

Branches and cases not covered

- `is_positive_number`: `"0"`, `""`, `" 3 "` (spaces), `"1.2.3"` and `".5"` are untested.
- `all_positive_numbers`: the empty list `[]` is untested.

- `compute_percent_yield`: there is no test over 100, and none for a rounded value like `1, 3` giving `33.33`.
- `find_limiting_reactant`: the tie case, where the first name should win, is untested.
- `smallest_ratio`: your one test only exercises the branch where reactant two is smaller. It needs a case where reactant one is smaller, and a tie.
- `compute_theoretical_yield`: your one test has equal coefficients. It needs a case like 1:2 and a case that needs rounding.
- `history_lines`: there is no test with a matching type, several entries, or an empty list.
- `calculate_percent_yield`: it needs tests for the state values being set, the over-100% message, and a zero theoretical yield being rejected.
- `calculate_limiting_reactant`: there are no invalid-number tests, no blank-name test, and no check that the history entry was saved or that names are trimmed.
- `calculate_theoretical_yield`: there is no invalid-input test and no check of the history entry.
- `filter_history`: it needs a test with some saved calculations, ideally through `history_lines` and `history_page`.

The grading says “valid and thorough,” so I’d close at least the helper gaps and the invalid-input cases for each calculate route. Do you want me to add these tests to `chemcalc.py`? Since a failing test could stop the site from deploying, I’d keep them to expected values I’ve worked out by hand.

can u add unit tests to make this requriemtn met

I replaced the test section in `chemcalc.py`, and every helper and route now has tests. I ran all of them against a small stand-in for Drafter, and none failed. That stand-in isn’t the real library, so run the file once in your environment too. Your installed Drafter still needs to accept `.state` on the returned `Page`, since the route tests read it.

What the new tests cover

- Helpers: `is_positive_number` gets zero, blank, spaces, `.5`, two decimal points and letters. `all_positive_numbers` gets the empty list. The two ratio and limiting-reactant functions get each branch plus a tie. `compute_percent_yield` gets a value over 100 and a rounding case. `compute_theoretical_yield` gets a 1:2 ratio and a rounding case.

- Previously untested functions: `add_calculation`, `history_lines` with a filter match, no match and an empty list, `page_buttons`, `nav_buttons`, `error_page` and `history_page`.
- Calculate routes, valid input: each one checks that the state fields change and that the saved `Calculation` has the right type, result and description. This includes names being trimmed, the over-100% case, and the theoretical yield using a limiting reactant found earlier.
- Calculate routes, invalid input: letters, zero, blank boxes and blank names are rejected. Each rejected case checks that the history is still empty and the state fields haven't changed.
- Page routes and `filter_history`: these check `current_page` and that the history is kept, now with a non-empty list.

I wrote two small helper functions for the tests, `blank_state()` and `sample_history()`, and they have docstrings like the rest. The tests use loops over bad-input cases, so keep a comment there if your TA asks about them. The file is still free of lines over 100 characters.

Chemcalc

PY

main.py:501: error: List item 0 has incompatible type "BulletedList"; expected "str" [list-item]
Found 1 error in 1 file (checked 1 source file)
Error: Process completed with exit code 1.

Edited a file, ran a command

Mypy was inferring the type of `body` from its first assignment, a list of strings, and then rejected the second assignment, which put a `BulletedList` in it. I changed `body` in `history_page` to a list annotated as `list`, which can hold both text and components. I also made the empty-history message the default value, with the `BulletedList` replacing it only when there are saved calculations.

The behavior is the same, and the tests still pass against my stand-in Drafter. Mypy isn't installed here, so I couldn't rerun it. The other list concatenations in the file don't use that pattern, so I don't expect further errors of this kind, but I can't promise that until you rerun your checker. If it reports another error, paste it and I'll fix it.

The line number in your error is from your copy named `main.py`, so the change is in `chemcalc.py`. Copy over the whole file or just the `history_page` lines.

can u fix it

Ran 2 commands, shared files

I fixed it a different way this time. `history_page` in `chemcalc.py` no longer builds a list that mixes text and a `BulletedList`, which was what mypy complained about. If nothing is saved yet, it now sets `lines` to a one-item list containing “No calculations to show yet.” It then always shows `BulletedList(lines)`, so the empty-history message appears as a single bullet instead of plain text.

My tests still pass against my stand-in for Drafter. Mypy isn’t installed here, so I couldn’t run it myself. Copy the whole updated file over your `main.py`, not just that function, and rerun your checker. If it reports another error, paste it here.